Introducción a las Redes Neuronales Convolucionales (CNN)

Minería de datos
Universidad de Murcia

Curso 2025/2026

Índice

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

Contexto y motivación

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

¿Qué es una imagen para una red neuronal?

- Una imagen es una **rejilla** de valores: píxeles ordenados espacialmente.
- Representación típica (RGB):

$$X \in \mathbb{R}^{H \times W \times C}, \quad C = 3$$

- Propiedades clave: **localidad**, **estructura espacial**, **patrones repetibles**.

Tareas típicas en Visión por Computador

- **Clasificación**: una etiqueta por imagen (gato/perro).
- **Detección**: cajas + etiquetas (dónde está cada objeto).
- **Segmentación**: etiqueta por píxel (carretera, cielo, coche...).
- **Super-resolución** / **denoising** / **restauración**.
- **Reconocimiento facial**, **OCR**, **medicina**, **satélite**, etc.

Recordatorio: MLP (feedforward) aplicado a imágenes

- Una MLP recibe un **vector** de entrada: hay que **aplanar** la imagen.
- Si $\mathsf{X} \in \mathbb{R}^{H \times W \times C}$, se convierte en
$$\mathsf{x} \in \mathbb{R}^{HWC}.$$
- Luego se aplican capas densas:
$$\mathsf{h}^{(l)} = \phi(\mathsf{W}^{(l)}\mathsf{h}^{(l-1)} + \mathsf{b}^{(l)}).$$

Limitación 1 de MLP: explosión de parámetros

- Ejemplo: imagen $224 \times 224 \times 3 \Rightarrow HWC = 150{,}528$.
- Una capa densa a 1024 neuronas:

$$\#\text{pesos} = 150{,}528 \times 1024 \approx 154 \text{ millones.}$$

- Consecuencias:
  - mucha memoria y coste computacional,
  - más riesgo de **sobreajuste**,
  - difícil entrenar con pocos datos.

Limitación 2 de MLP: pierde estructura espacial

- Al aplanar, se destruye la noción de **vecindad**.
- Un píxel y su vecino (a 1 unidad) quedan tan "lejos" como cualquier otro en el vector.
- Las MLP no incorporan de serie:
  - **localidad** (patrones pequeños como bordes),
  - **invariancias/equivariancias** (traslaciones, etc.),
  - **composición jerárquica** (bordes $\rightarrow$ texturas $\rightarrow$ partes $\rightarrow$ objetos).

¿Qué sesgos inductivos queremos?

Idea

En visión, queremos que el modelo **aproveche** la estructura de los datos.

- **Localidad**: los patrones útiles suelen ser locales.
- **Compartición de pesos**: el mismo patrón puede aparecer en cualquier posición.
- **Equivariancia a traslación**: si la entrada se desplaza, las activaciones se desplazan.
- **Jerarquía**: combinar patrones simples en otros más complejos.

Idea clave de las CNN

- Sustituimos conexiones totalmente densas por:
  - **conectividad dispersa** (cada neurona ve un *patch*),
  - **filtros convolucionales** (kernels),
  - **compartición de pesos** a lo largo de la imagen.
- Resultado: menos parámetros + mejor inductive bias para datos en rejilla.

Un poco de historia (muy breve)

- Inspiración biológica: **campos receptivos** en la corteza visual.
- **LeNet-5** (años 90): dígitos y documentos.
- **AlexNet (2012)**: salto en ImageNet (GPU + ReLU + data augmentation).
- **ResNet (2015)**: redes muy profundas con conexiones residuales.
- Hoy: CNNs muy eficientes + modelos híbridos (CNN + atención).

¿Cuándo usar CNN? ¿Y cuándo no?

- CNN: datos con estructura de **rejilla** (imágenes, espectrogramas, señales 1D, vídeo 3D).
- Ventajas: generalizan bien con menos parámetros, aprenden características jerárquicas.
- Limitaciones típicas:
  - capturan peor dependencias globales si no son suficientemente profundas,
  - sensibilidad a rotaciones/escala si no se entrena con aumentos adecuados,
  - coste alto en alta resolución sin trucos (downsampling, patching, etc.).

Convolución: conceptos básicos

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

Convolución: la idea

Definicion

Es una operación especial que se aplica a una matriz concreta (normalmente la matriz de imagen) utilizando otra matriz (normalmente la matriz de filtro). La operación consiste en multiplicar los valores de una celda correspondiente a una fila y columna concretas de la matriz de imagen por el valor de la celda correspondiente de la matriz de filtro. Hacemos esto con los valores de todas las celdas dentro del intervalo de la matriz de filtro y los sumamos para formar una salida.

Convolución: la idea

Definicion

Es una operación especial que se aplica a una matriz concreta (normalmente la matriz de imagen) utilizando otra matriz (normalmente la matriz de filtro). La operación consiste en multiplicar los valores de una celda correspondiente a una fila y columna concretas de la matriz de imagen por el valor de la celda correspondiente de la matriz de filtro. Hacemos esto con los valores de todas las celdas dentro del intervalo de la matriz de filtro y los sumamos para formar una salida.

En deep learning, normalmente se usa **correlación cruzada** (sin invertir el kernel):

$$y[t] = \sum_{k} w[k]\, x[t+k]$$

Convolución 2D: definición

- Entrada (una imagen o feature map): $X \in \mathbb{R}^{H \times W}$
- Kernel: $K \in \mathbb{R}^{k_h \times k_w}$
- Salida:

$$Y[i,j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K[u,v]\, X[i+u,\, j+v]$$

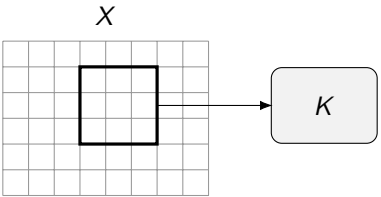


Ilustración visual: entrada, kernel y salida (VALID)

Entrada (toy 5×5)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

Kernel (3×3)

−1	0	1
−2	0	2
−1	0	1

Salida (3×3)

- Deslizamos el kernel; cada posición produce un valor de salida (`valid` ⇒ sin padding).

Ilustración (paso 1): posición (fila 0, col 0)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

4		

$$y_{0,0} = \sum_{u,v} x_{0+u,\,0+v}\; w_{u,v} \quad \Rightarrow \quad y_{0,0} = 4$$

Ilustración (paso 2): posición (fila 0, col 1)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

	4	

$$y_{0,1} = 4$$

Ilustración (paso 3): posición (fila 0, col 2)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

		0

$$y_{0,2} = 0$$

Ilustración (paso 4): posición (fila 1, col 0)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

4		

$$y_{1,0} = 4$$

Ilustración (paso 5): posición (fila 1, col 1)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

	4	

$$y_{1,1} = 4$$

Ilustración (paso 6): posición (fila 1, col 2)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

		0

$$y_{1,2} = 0$$

Ilustración (paso 7): posición (fila 2, col 0)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

4		

$$y_{2,0} = 4$$

Ilustración (paso 8): posición (fila 2, col 1)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

	4	

$$y_{2,1} = 4$$

Ilustración (paso 9): posición (fila 2, col 2)

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

-1	0	1
-2	0	2
-1	0	1

		0

$$y_{2,2} = 0$$

Ilustración: feature map completo (VALID, stride=1)

Entrada

0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

Salida

4	4	0
4	4	0
4	4	0

- El filtro responde fuerte cerca del **borde vertical** (transición 0→1).

Ilustración: efecto del stride

- Si usamos **stride** $S = 2$, el kernel salta posiciones $\Rightarrow$ salida más pequeña.

$$H_{\text{out}} = \left\lfloor \frac{H - K}{S} \right\rfloor + 1$$

Entrada (5×5)

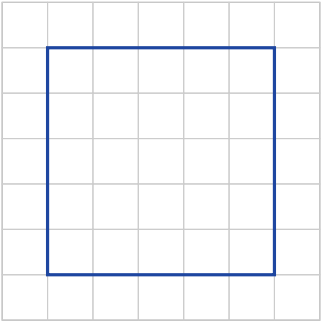
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

Salida (2×2) con $S = 2$

4	0
4	0

Ilustración: padding (`same`)

- Problema: con `valid` perdemos tamaño en bordes.
- Solución: **zero-padding** para mantener resolución (`same`).



`same`: añadimos borde de ceros

Los pesos del filtro determinan la "feature"

- Los pesos w_i del kernel determinan qué patrón detecta la capa.
- Ejemplo: filtros de Sobel (detección de bordes).

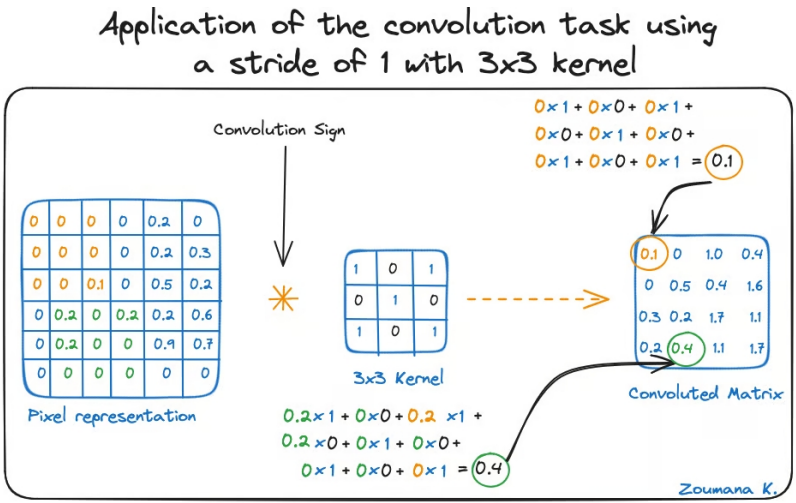
Sobel horizontal

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

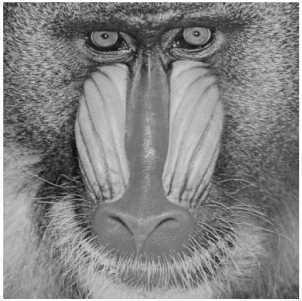
Sobel vertical

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Ejemplo


Application of the convolution task using a stride of 1 with 3x3 kernel
Convolution Sign
0 0 0 0 0.2 0
0 0 0 0 0.2 0.3
0 0 0.1 0 0.5 0.2
0 0.2 0 0.2 0.2 0.6
0 0.2 0 0 0.9 0.7
0 0 0 0 0 0
Pixel representation
1 0 1
0 1 0
1 0 1
3x3 Kernel
0.2x1 + 0x0 + 0.2 x1 +
0.2 x0 + 0x1 + 0x0 +
0x1 + 0x0 + 0x1 = 0.4
0x1 + 0x0 + 0x1 +
0x0 + 0x1 + 0x0 +
0x1 + 0x0 + 0x1 = 0.1
0.1 0 1.0 0.4
0 0.5 0.4 1.6
0.3 0.2 1.7 1.1
0.2 0.4 1.1 1.7
Convoluted Matrix
Zoumana K.

Ejemplo



Horizontal edge

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Vertical edge

$$\begin{bmatrix} -1 & 0 & 1 \\ 2 & 0 & -2 \\ -1 & 0 & 1 \end{bmatrix}$$

Ejemplo

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Visualization of a curve detector filter

Ejemplo

Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

$*$

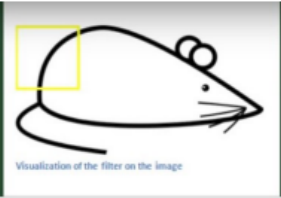
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = 0

Image 1: An example with result 0, thus confirming absence of the feature.

Ejemplo



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = (50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600 (A large number!)

Image 2: An example with a large value as result, thus confirming the presence of the filter

Dos enfoques comunes: `VALID` vs `SAME`

`VALID`

- Solo computa posiciones donde el kernel cabe entero.
- Tamaño (1D): $m - |w| + 1$
- En 2D: reduce altura y anchura.

`SAME`

- Mantiene tamaño de salida (con $S = 1$).
- Requiere valores fuera del soporte $\Rightarrow$ **zero-padding**.

Convolución 2D con múltiples filtros (salida con profundidad)

- En una capa solemos aplicar K filtros: $\{w^{(1)}, \ldots, w^{(K)}\}$.
- Obtenemos K mapas y los apilamos:

$$u^{(\ell+1)} = \left[\, u^{(\ell)} * w^{(1)},\ \ldots,\ u^{(\ell)} * w^{(K)} \,\right] \in \mathbb{R}^{H \times W \times K}$$

- El siguiente kernel debe tener profundidad K:

$$(u * w)_{y,x} = \sum_{i,j,k} u(i,j,k)\ w(y-i, x-j, k)$$

Capas consecutivas: “canales” como respuestas a filtros

- Cada capa produce un **volumen** $H \times W \times C$:
  - C canales $\approx$ respuestas a distintos filtros aprendidos.
- Al apilar capas:
  - aumenta la riqueza de características,
  - cambian resoluciones y profundidades,
  - se construyen representaciones jerárquicas.

$$\boxed{H \times W \times 3} \xrightarrow{\text{conv}} \boxed{H \times W \times 32} \xrightarrow{\text{downsample}} \boxed{\frac{H}{2} \times \frac{W}{2} \times 64} \xrightarrow{\text{conv}} \boxed{\frac{H}{4} \times \frac{W}{4} \times 128}$$

Convolución con múltiples canales

- Una imagen RGB: $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, con $C_{\text{in}} = 3$.
- Un filtro produce **un canal** de salida y combina todos los canales de entrada:

$$K \in \mathbb{R}^{k_h \times k_w \times C_{\text{in}}}$$

- Si tenemos C_{out} filtros:

$$Y \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}}$$

Por qué funciona: conectividad dispersa y pesos compartidos

MLP (densa)

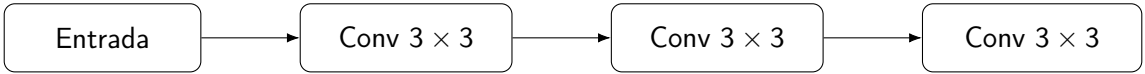
- cada salida depende de **todos** los píxeles
- parámetros crecen como $O(HWC \cdot n)$

CNN (conv)

- cada salida depende de un **patch** local
- el mismo kernel se usa en todas las posiciones
- parámetros $\approx k_h k_w C_{\text{in}} C_{\text{out}}$

Campo receptivo (*receptive field*)

- El **campo receptivo** de una neurona es la región de entrada que puede afectarla.
- En una CNN, el campo receptivo **crece** al apilar capas.
- Con kernels pequeños (p.ej. 3×3) se puede alcanzar contexto amplio con profundidad.



RF crece: $3 \rightarrow 5 \rightarrow 7 \rightarrow \ldots$

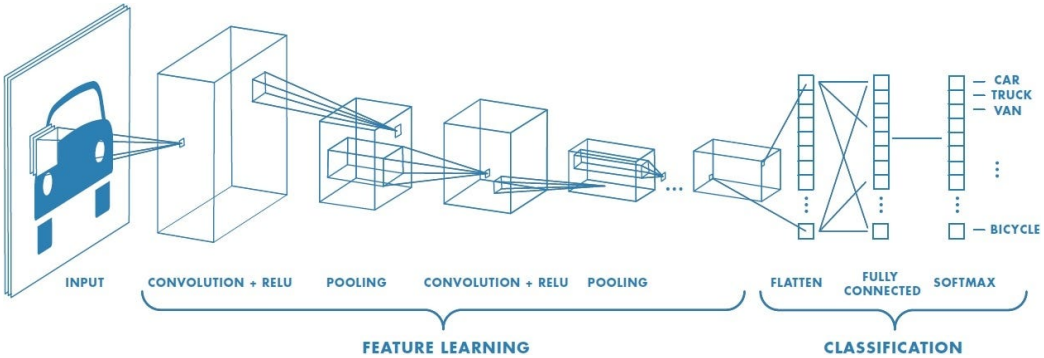
¿Qué aprende un filtro? (intuición)

- Capas iniciales: bordes, orientaciones, blobs de color.
- Capas medias: texturas, patrones repetitivos.
- Capas profundas: partes de objetos y configuraciones.
- **Mapas de activación**: "dónde" se detecta un patrón en la imagen.

Bloques de una CNN

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

Idea


CAR
TRUCK
VAN
BICYCLE
INPUT
CONVOLUTION + RELU
POOLING
CONVOLUTION + RELU
POOLING
FLATTEN
FULLY CONNECTED
SOFTMAX
FEATURE LEARNING
CLASSIFICATION

¿De qué partes se compone una CNN?

Idea general

Una **convolutional neural network (CNN)** se construye típicamente con **cuatro bloques principales**.

- **Convolutional layers** (capas convolucionales)
- **ReLU** (Rectified Linear Unit)
- **Pooling layers** (capas de pooling)
- **Fully connected layers** (capas totalmente conectadas)

Pero... ¿cómo "aprenden" las CNN con esos bloques?

- Estos componentes ayudan a las CNN a **extraer patrones** y **características** de una imagen.
- Intuición: imitan (a alto nivel) cómo el cerebro humano procesa **patrones visuales**.
- La idea clave es construir una representación **jerárquica**:

$$\text{bordes} \rightarrow \text{curvas/texturas} \rightarrow \text{partes} \rightarrow \text{objeto}$$

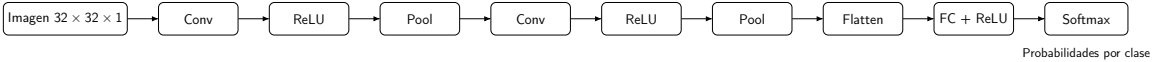
En esta sección

Definimos cada bloque usando un ejemplo clásico: **clasificación de un dígito manuscrito**.

Ejemplo guía: clasificación de un dígito manuscrito

- Entrada: imagen en escala de grises (por ejemplo 32×32).
- Salida: una etiqueta entre $\{0, 1, \ldots, 9\}$.
- La CNN aprende filtros para detectar rasgos:
  - curvas del dígito, bordes, trazos,
  - configuración global de la forma,
  - patrones más abstractos en capas profundas.

Arquitectura típica para reconocimiento de dígitos



Capas convolucionales: primer bloque fundamental

- La **convolution layer** aplica una operación llamada **convolución** sobre la imagen.
- La convolución usa una **ventana deslizante** sobre una matriz de píxeles.
- Esa ventana se llama **kernel** o **filter** (términos intercambiables).

Intuición

Un kernel es una "lupa" pequeña que recorre la imagen buscando un patrón concreto.

¿Qué hacen varios filtros en una misma capa?

- En una capa conv se aplican **varios filtros del mismo tamaño**.
- Cada filtro puede especializarse en detectar:
  - bordes, esquinas,
  - curvas (trazos del dígito),
  - texturas simples,
  - y, con más capas, formas más abstractas.
- Cada filtro produce un **feature map** (mapa de activación).

Explicación en lenguaje simple

- Usamos pequeñas rejillas (kernels) que se mueven sobre la imagen.
- Cada rejilla resalta dónde aparece su patrón (líneas, curvas, etc.).
- El resultado es una nueva rejilla:
  - valores altos donde el patrón está presente,
  - valores bajos donde no aparece.

Consecuencia

Con varios filtros, la CNN obtiene una visión rica de los patrones que componen la imagen.

Ejemplo: imagen 32×32 en escala de grises

- Consideramos una imagen de dígito manuscrito:

$$X \in \mathbb{R}^{32 \times 32}$$

- Cada entrada $X[i, j]$ es la intensidad del píxel (0–255 o normalizado 0–1).
- En clase, a menudo se muestran los valores como una matriz para ilustrar el proceso.

Representación

Imagen $\leftrightarrow$ matriz de píxeles.

Kernel 3×3: pesos y significado

- Un kernel típico:

$$W \in \mathbb{R}^{3\times 3}$$

- Sus entradas son **pesos** (parámetros) que determinan qué patrón se detecta.
- Visualmente, a veces se representan:
  - ceros en celdas negras,
  - unos (u otros valores) en celdas blancas.

Pregunta clave: ¿hay que elegir esos pesos a mano?

En la práctica, los pesos de los kernels se **aprenden durante el entrenamiento**.

- Al inicio: pesos aleatorios (o inicialización tipo He).
- Durante entrenamiento:
  - se calcula la pérdida,
  - se aplica backprop,
  - se actualizan los kernels para mejorar la tarea (p.ej. clasificación).

Kernel como *feature detector*

- En muchos textos, kernel = **feature detector**.
- Porque los pesos pueden ajustarse para detectar rasgos específicos.

Ejemplos clásicos

- Kernel que promedia vecinos $\Rightarrow$ **blur** (suavizado).
- Kernel que resta vecinos $\Rightarrow$ **detección de bordes**.

Más capas convolucionales $\Rightarrow$ features más abstractas

- Capas iniciales: patrones simples (bordes, orientaciones).
- Capas intermedias: texturas y combinaciones (curvas, esquinas).
- Capas profundas: estructuras de alto nivel (forma del dígito, partes).

Idea

Profundidad construye jerarquía de representación.

Función de activación: ReLU

- Tras la convolución, aplicamos una activación no lineal.
- La más común: **ReLU** (Rectified Linear Unit).

$$\mathrm{ReLU}(x) = \text{máx}(0, x)$$

¿Por qué ReLU?

- introduce **no linealidad** $\Rightarrow$ relaciones complejas,
- ayuda con el **vanishing gradient** (comparado con sigmoide/tanh).

Pooling layer: objetivo

- El pooling selecciona las características más relevantes del feature map.
- Reduce la dimensión espacial $\Rightarrow$
  - menos memoria,
  - menos coste computacional,
  - puede ayudar a mitigar **overfitting**.

Idea

Compresión espacial conservando lo importante.

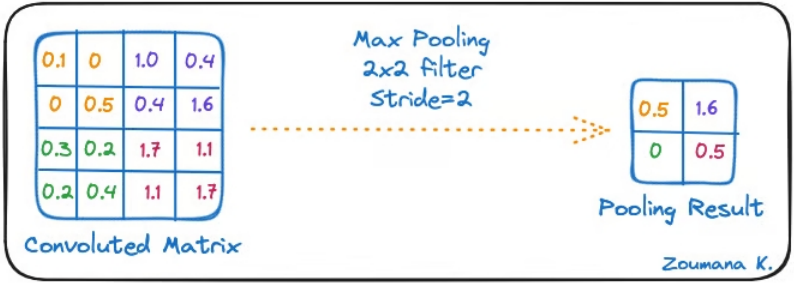
Tipos de pooling más comunes

- **Max pooling**: toma el máximo de cada región.
- **Sum pooling**: suma los valores (menos usado hoy).
- **Average pooling**: promedio de la región.

Caso típico

MaxPool 2×2 con stride 2 $\Rightarrow$ reduce H, W a la mitad.

Ejemplo


Application of max pooling with
a stride of 2 using 2x2 filter
0.1 0 1.0 0.4
0 0.5 0.4 1.6
0.3 0.2 1.7 1.1
0.2 0.4 1.1 1.7
Convoluted Matrix
Max Pooling
2x2 filter
Stride=2
0.5 1.6
0 0.5
Pooling Result
Zoumana K.

Efecto del pooling en el tamaño

- El feature map se hace más pequeño a medida que aplicamos pooling.
- Esto fuerza a la red a conservar solo información robusta a pequeñas traslaciones.

$$\boxed{H \times W \times C} \xrightarrow{\text{MaxPool } 2 \times 2,\ S = 2} \boxed{\frac{H}{2} \times \frac{W}{2} \times C}$$

Flatten: conectar con el clasificador

- Tras el último bloque conv/pooling tenemos un volumen $H' \times W' \times C'$.
- Para alimentar capas densas se suele hacer:

$$\text{Flatten} : \mathbb{R}^{H' \times W' \times C'} \to \mathbb{R}^{H'W'C'}$$

- Alternativa moderna: **Global Average Pooling** (a veces mejor y con menos parámetros).

Fully connected layers: clasificación final

- Las capas totalmente conectadas están al final de la CNN.
- Entrada: vector 1D (flatten) desde el último pooling.
- Suelen llevar activación ReLU para no linealidad.

Rol

Combinar las features extraídas para decidir la clase.

Capa Softmax: probabilidades por clase

- Última capa típica en clasificación multiclase: **softmax**.

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

- Produce probabilidades para cada etiqueta (p.ej. $K = 10$ dígitos).
- Predicción final:

$$\hat{y} = \arg\max_k \hat{p}_k$$

Resumen del flujo completo

- **Convolución**: detecta patrones locales con kernels aprendidos.
- **ReLU**: introduce no linealidad y ayuda a entrenar redes profundas.
- **Pooling**: reduce dimensión y retiene lo más relevante.
- **FC + Softmax**: combinan features y producen probabilidades por clase.

Regularización: Dropout (y cuándo usarlo)

- Dropout: apaga neuronas aleatoriamente durante entrenamiento.
- En CNNs modernas:
  - suele usarse poco dentro de bloques convolucionales,
  - es común en la parte final (clasificador) o como *spatial dropout*.
- Alternativas muy efectivas: **data augmentation**, **weight decay**.

¿Por qué hablar de "familias" de CNN?

- Distintas familias responden a distintos objetivos:
  - **precisión** (modelos grandes),
  - **profundidad entrenable** (estabilidad),
  - **eficiencia** (latencia/memoria),
  - **tareas** (clasificación vs segmentación).
- Cada familia propone un **bloque** o **patrón arquitectónico** reutilizable.

Idea

Cambian el *sesgo inductivo* y el coste computacional sin cambiar el objetivo: aprender features jerárquicas.

Ejes de diseño (visión de alto nivel)

Capacidad

- **Profundidad**: más composición/abstracción
- **Anchura** (canales): más variedad de features.
- **Resolución**: más detalle espacial.

Coste

- FLOPs y latencia.
- Memoria (activaciones + parámetros).
- Escalabilidad a edge/móvil.

Trade-off

Más precisión suele implicar más coste; familias como MobileNet buscan "lo mejor posible" bajo restricciones.

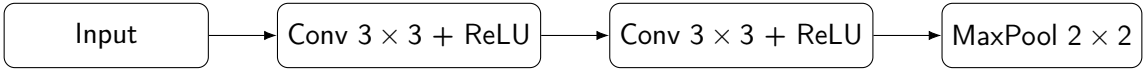
VGG: idea central

- Arquitectura **uniforme** y fácil de entender:
  - repite muchas convoluciones 3×3,
  - aplica downsampling cada ciertos bloques,
  - al final: clasificación.
- Intuición: con suficientes capas 3×3 se obtiene un gran campo receptivo.

Por qué es importante

Sirve como **baseline** conceptual: "profundidad + simplicidad".

VGG: bloque típico



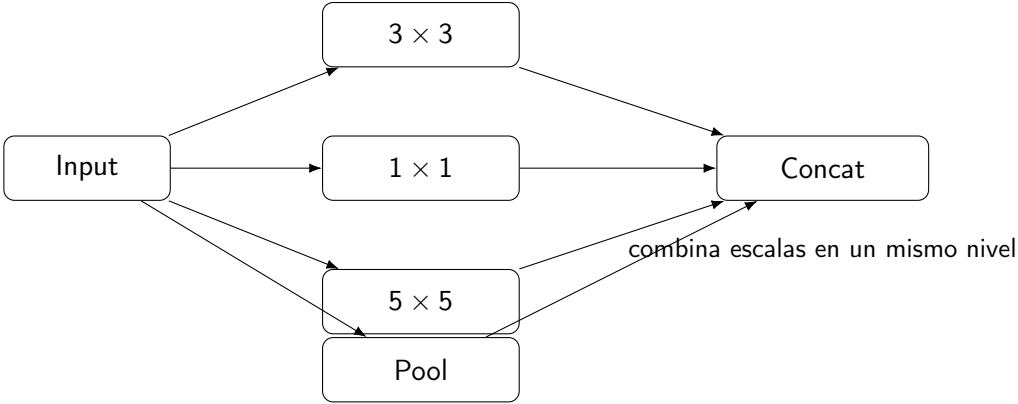
Repetir $\Rightarrow$ más canales, menos resolución

- Pros: diseño simple, muy didáctica.
- Contras: pesada en parámetros/cómputo (especialmente con FC).

Inception: multi-escala en paralelo

- Problema: un único tamaño de kernel puede ser subóptimo.
- Solución: **ramas en paralelo** que capturan patrones a distintas escalas:
  - 1×1 (mezcla de canales / compresión),
  - 3×3, 5×5 (o factorizadas),
  - pooling.
- Al final: concatenación de canales.

Inception: diagrama del módulo



- Pros: buen rendimiento con eficiencia cuidadosa.
- Contras: bloque más complejo de implementar/optimizar.

ResNet: la revolución de las conexiones residuales

- Problema: entrenar redes muy profundas puede ser inestable.
- Idea: aprender una **corrección** sobre la identidad:

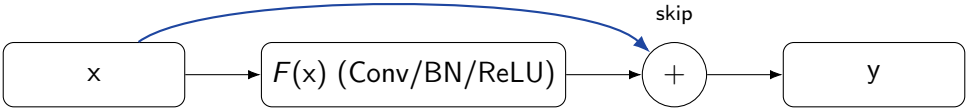
$$y = x + F(x)$$

- Si $F(x) \approx 0$, el bloque se comporta como identidad $\Rightarrow$ facilita flujo de gradientes.

Intuición

"Que sea fácil ser una red simple; y si hace falta, añadir complejidad".

ResNet: bloque residual

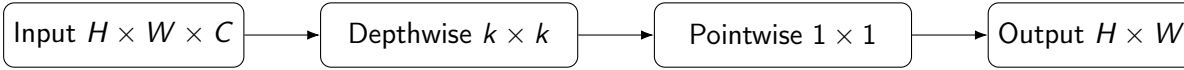


- Pros: permite redes muy profundas y robustas.
- Uso típico: backbone estándar en clasificación/detección/segmentación.

MobileNet: eficiencia para móvil/edge

- Objetivo: reducir coste manteniendo precisión razonable.
- Núcleo: **depthwise separable convolution**:
  - **Depthwise**: un kernel $k \times k$ por canal.
  - **Pointwise**: conv 1×1 para mezclar canales.
- Reduce operaciones respecto a conv estándar.

MobileNet: bloque separable (intuición)



- Pros: baja latencia/energía; ideal en dispositivos.
- Contras: puede perder precisión frente a backbones grandes.

U-Net: diseñada para segmentación

- Tarea: asignar una etiqueta **por píxel** (más exigente que clasificar).
- Arquitectura **encoder–decoder**:
  - encoder: reduce resolución y extrae semántica,
  - decoder: recupera resolución para predecir máscaras.
- Clave: **skip connections** para recuperar detalle espacial.

U-Net: diagrama (encoder–decoder + skips)

- Pros: excelente precisión espacial (muy usada en imagen médica).
- Contras: más memoria por skips/activaciones.

Comparativa rápida: ¿cuándo usar cuál?

Familia	**Idea**	**Uso típico**
VGG	Bloques 3×3 repetidos	baseline/didáctica
Inception	Multi-escala en paralelo	clasificación eficiente
ResNet	Residual (skip)	backbone general
MobileNet	Separable conv	móvil/edge
U-Net	Encoder–decoder + skips	segmentación

Resumen

- Las familias de CNN son **patrones** que equilibran precisión y coste.
- VGG: simple y profundo; Inception: multi-escala; ResNet: entrenabilidad.
- MobileNet: eficiencia; U-Net: salida densa (por píxel).

Entrenamiento y buenas prácticas

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

Objetivo, pérdidas y métricas

- Clasificación multiclase (típico):

$$\mathcal{L}(\mathrm{y}, \hat{\mathrm{y}}) = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$$

- Métricas habituales: accuracy, top-k, F1 (si hay desbalanceo).
- Segmentación: IoU, Dice, BCE+Dice, etc.

Optimización: lo que suele funcionar

- Inicialización tipo **He** (para ReLU).
- Optimizadores:
  - **SGD + momentum**: muy robusto (a menudo mejor final).
  - **Adam**: converge rápido (muy buen baseline).
- Learning rate: el hiperparámetro más crítico.
- Trucos: scheduler (cosine/step), warmup, weight decay.

Regularización y *data augmentation*

- **Data augmentation** (casi imprescindible):
  - flips, crops, rotaciones pequeñas,
  - jitter de color, blur,
  - MixUp / CutMix (avanzado).
- **Weight decay** (L2) y early stopping.
- BatchNorm suele actuar como regularizador adicional.

Transfer learning: receta práctica

- Usar una CNN pre-entrenada (p.ej. en ImageNet) como **backbone**.
- Estrategia común:
  1. congelar capas iniciales,
  2. entrenar "cabeza" (clasificador),
  3. descongelar parcialmente y **fine-tuning** con LR pequeño.
- Muy útil si tu dataset es pequeño/mediano.

Ejemplo mínimo (Keras): CNN para CIFAR-10

```
import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, 3, padding="same"), layers.ReLU(),
    layers.Conv2D(32, 3, padding="same"), layers.ReLU(),
    layers.MaxPool2D(),

    layers.Conv2D(64, 3, padding="same"), layers.ReLU(),
    layers.Conv2D(64, 3, padding="same"), layers.ReLU(),
    layers.MaxPool2D(),

    layers.GlobalAveragePooling2D(),
    layers.Dense(10, activation="softmax")
])

model.compile(optimizer="adam",
```

Checklist: errores comunes en CNNs

- Normalización de entrada mal aplicada (mean/std incorrectos).
- Data augmentation excesivo (introduce ruido dañino).
- **Overfitting**: train sube, val baja (solución: aug, WD, menos capacidad).
- BatchNorm con batches muy pequeños (puede ser inestable).
- Métrica inapropiada (desbalanceo $\Rightarrow$ accuracy engañosa).

Aplicaciones y cierre

1. Contexto y motivación
2. Convolución: conceptos básicos
3. Bloques de una CNN
4. Clases de CNN
5. Entrenamiento y buenas prácticas
6. Aplicaciones y cierre

CNN más allá de imágenes

- **1D-CNN**: señales (sensores, audio), series temporales (con ventanas).
- **2D-CNN**: imágenes, mapas tiempo-frecuencia (espectrogramas).
- **3D-CNN**: vídeo (tiempo como tercera dimensión).
- Combinaciones:
  - CNN (features) + RNN/Transformer (secuencia),
  - CNNs en el backbone de detección/segmentación.

Resumen

- Las CNN solucionan las limitaciones clave de las MLP en visión: **localidad + pesos compartidos + jerarquía**.
- Componentes: conv, activación, (pool/stride), normalización, regularización.
- Arquitecturas: de LeNet/AlexNet a ResNet, U-Net y modelos eficientes.